

Contents

Pet Store Agentic — Architecture & Design Principles	1
1. External framework integration — the Adapter discipline	1
2. Fan-out authority is exclusive to K9-AIF	2
3. Substrate neutrality — the ABB/SBB swap	2
4. The deterministic/agentic boundary is architecture, not a performance detail	2
5. Governance lives in the harness, not the prompt	2
6. Verify-before-build	2
Document map	3

Pet Store Agentic — Architecture & Design Principles

This file captures the *general* principles this project exists to demonstrate — the reasoning behind decisions in `project.md`, generalized beyond this one domain. `project.md` is the concrete build spec; this is why it's built that way. `CLAUDE.md` holds project-specific invariants; `PLAN.md` holds build status. This is the one of the four that should still be true if the domain were insurance claims instead of pet supplies.

1. External framework integration — the Adapter discipline

Principle: any external agent framework is wrapped via `BaseAdapter` multiple inheritance, at whichever K9-AIF layer matches that framework's *natural unit of encapsulation* — not at whatever layer looks most impressive.

Pet Store Agentic does not use CrewAI. No CrewAI dependency exists anywhere in this project. The precedent below is cited purely to justify *the pattern* — it's an existing, already-shipped example of the same design discipline applied to a different external framework, in `k9_aif_abb/k9_adapters/crewai/crewai_orchestrator_adapter.py` elsewhere in the K9-AIF framework repo:

```
class CrewAIOrchestratorAdapter(BaseOrchestrator, BaseAdapter):
    def adapt_input(self, payload: dict) -> dict: ...    # K9-AIF payload -> CrewAI's shape
    def adapt_output(self, result: Any) -> dict: ...    # CrewAI's result -> K9-AIF's shape
    def execute_flow(self, payload: dict) -> dict: ...  # calls crew.kickoff() / crew.run()
```

CrewAI's `Crew` is *itself* a multi-agent orchestrating construct — it has its own internal agents and its own `kickoff()`. So the natural wrapping point is the **Orchestrator** layer: `CrewAIOrchestratorAdapter` is simultaneously a valid `BaseOrchestrator` (K9-AIF sees a normal orchestrator) and an explicit `BaseAdapter` (the type system says “this bridges to something external”).

The Claude Agent SDK's unit of encapsulation is different — one session, one autonomous tool loop, not a crew. So it wraps one layer down, at the **Agent** layer:

```
class SdkDiagnosisAgent(DiagnosisAgent, BaseAdapter):
    def adapt_input(self, request: DiagnosisRequest) -> dict: ...    # -> SDK session input
    def adapt_output(self, sdk_result: Any) -> DiagnosisResult: ...  # SDK output -> DiagnosisResult
    def diagnose(self, request: DiagnosisRequest) -> DiagnosisResult: ...
```

Same shape, same multiple-inheritance discipline, same `adapt_input/adapt_output` pair — just one layer lower, because that's where the SDK's own unit of work actually sits.

Rule of thumb for the next external framework this happens to: ask what its natural unit of encapsulation is (a crew, a single agent session, a single tool call) before deciding which K9-AIF layer it should extend alongside `BaseAdapter`.

2. Fan-out authority is exclusive to K9-AIF

Only the K9-AIF hierarchy — Orchestrator → Squad → Agent — may fan out or delegate. When an external framework has its own internal delegation concept (the Agent SDK’s subagents; potentially CrewAI’s own internal task delegation), that mechanism is disabled or bypassed at the wrapping boundary.

Why this is non-negotiable, not a style preference: provenance is a graph (DELEGATES_TO edges) built from what K9-AIF can see. If a wrapped framework quietly delegates further on its own, K9-AIF records “one agent ran” when three sessions actually ran underneath it. The graph doesn’t error — it just silently under-reports, and nobody notices until they query it expecting a complete answer. Two competing delegation trees means one of them isn’t in the graph at all. There is no such thing as governing an untracked delegation path, so the wrap must actively deny it, not just decline to use it.

3. Substrate neutrality — the ABB/SBB swap

Any genuinely uncertain capability is defined once as an abstract ABB contract. Multiple SBBs may satisfy it — SDK-backed, direct-API, CrewAI-backed, whatever comes next — selected entirely by config, never by a code change at the call site.

Substitutability means the contract holds, not that every substrate performs equally. `DirectApiDiagnosisAgent` handling open-ended narratives worse than `SdkDiagnosisAgent` is a real, documented tradeoff, not a flaw to hide. The claim being proven is narrower and more honest than “all substrates are equivalent”: it’s “the system keeps working, and keeps enforcing the same gates, no matter which one is behind the contract today.”

4. The deterministic/agentic boundary is architecture, not a performance detail

Deterministic handlers register before the agentic fallback. This ordering *is* the argument — most enterprise workflow doesn’t need an agent, and proving that requires the deterministic path be structurally first, not just fast.

This generalizes past Pet Store Agentic: any system built on K9-AIF should be able to answer, per capability, “why does this need an agent at all?” — and “because the steps aren’t knowable in advance” is the only acceptable answer. “It’s more impressive this way” is not.

5. Governance lives in the harness, not the prompt

Gates are enforced by framework-level mechanisms — hooks, middleware, explicit code branches — that query external state (a registry, a database) directly. They never rely on conversation context holding a fact, and they are never expressed as a system-prompt instruction alone.

A prompt is a request the model can be talked out of, forget under context pressure, or lose to compaction. A hook querying an external registry on every invocation can’t be argued with, because it isn’t part of the argument.

6. Verify-before-build

Installed package signatures win over any spec, including this one and `project.md`. When an external SDK’s actual API differs from what was assumed, the difference gets recorded (`DEVIATIONS.md`) and the spec adjusts — never the reverse. A missing capability is useful information; a fabricated stand-in corrupts the reference implementation’s value as a reference.

Document map

File	Answers
Architecture_Guide.md (this file)	<i>Why</i> — principles that would hold in any domain, not just pet supplies
project.md	<i>What</i> — the concrete Pet Store Agentic build spec
CLAUDE.md	<i>Rules</i> — project-specific invariants and load-bearing tests
PLAN.md	<i>Status</i> — build order, current phase, pre-build verification steps
diagrams/	<i>Pictures</i> — four PlantUML views of the architecture in <code>project.md</code>